

Checklist

Implementing (Internal) Platforms

1. Platform Anatomy

Management Plane

- ☐ Define how developers will interact with the platform.
- ☐ Provide an appropriate **self-service portal/UI**.
- ☐ Decide whether a **CLI** is needed for quick or developer-oriented tasks.
- ☐ Provide **management APIs** for programmatic access.
- ☐ Determine whether an **automation language** will be exposed to users.
- ☐ Support common user journeys rather than only CRUD-style resource manipulation.
- ☐ Allow teams to create or obtain base-platform accounts where required.
- ☐ Allow users to provision services using higher-level abstractions or predefined compositions.
- ☐ Provide an inventory of provisioned services and environments.
- ☐ Show whether environments comply with regulatory requirements.
- ☐ Show whether environments use the latest approved reference architecture.
- ☐ Avoid relying heavily on manual “ClickOps” for tasks that need to scale.

The management plane is the visible, self-service part of the platform and may include a portal, API, CLI, and automation language.

2. Control Plane

- ☐ Establish a mechanism for translating developer requests into provisioned services.
- ☐ Define how the platform records the **desired state** of resources.
- ☐ Implement reconciliation between desired state and actual state.

- ☐ Support creation, deletion, and updating of managed resources.
- ☐ Decide whether reconciliation will be implemented directly or delegated to tools such as Kubernetes or cloud automation systems.
- ☐ Determine whether the control plane will be **multi-tenant**.
- ☐ Calculate and monitor the operational **cost per tenant**.
- ☐ Implement user and identity management.
- ☐ Track application ownership.
- ☐ Connect identities to billing, monitoring, and other platform capabilities.
- ☐ Maintain a **service catalog**.
- ☐ Record metadata such as service names, pricing models, and bills of materials.
- ☐ Implement **service orchestration**.
- ☐ Track services created by the platform through identifiers, tags, or another reliable mechanism.

The control plane connects the management and services planes and continually reconciles declared configuration with actual state.

3. Services Plane

- ☐ Identify the **base services** provided by the underlying cloud or infrastructure platform.
- ☐ Identify required **third-party services**.
- ☐ Identify any **custom services** the platform team must develop.
- ☐ Define a service interface for custom components.
- ☐ Make custom-service metadata available to the service catalog.
- ☐ Ensure custom services can be managed by the control plane.
- ☐ Capture usage information needed for billing or chargeback.
- ☐ Establish a process for accepting and governing **user-contributed services**.
- ☐ Include relevant SDLC capabilities such as:
 - ☐ Source Code Management
 - ☐ Continuous Integration
 - ☐ Continuous Delivery
 - ☐ Code analysis
 - ☐ Test management
 - ☐ Secret management
 - ☐ Policy management
 - ☐ Release management
 - ☐ Observability
 - ☐ Infrastructure as Code

The services plane can contain base-platform services, third-party products, custom services, and community-contributed services.

4. IDP Integration

- ☐ Integrate the platform with enterprise **Identity and Access Management (IAM)**.
- ☐ Provide single sign-on where appropriate.
- ☐ Apply granular access controls.
- ☐ Implement an automation controller for platform workflows.
- ☐ Maintain a catalog containing services and their automation definitions.
- ☐ Implement providers that execute provisioning and configuration tasks.
- ☐ Maintain a repository of service and resource metadata.
- ☐ Integrate developer-facing services into the portal where useful.
- ☐ Ensure the control plane provides a consistent experience across otherwise separate tools.

The IDP example in the document combines portal/CLI/API access with an automation controller, providers, a service catalog, and a state repository.

5. Automation Templates

- ☐ Determine whether simple templates are sufficient for the platform's use cases.
- ☐ Define which values developers are allowed to provide.
- ☐ Provide sensible defaults for unspecified values.
- ☐ Override or restrict values that users should not modify.
- ☐ Expand simple inputs into more complex resource definitions where needed.
- ☐ Ensure generated output is valid for the underlying automation tool.
- ☐ Avoid exposing developers to unnecessarily long or confusing parameter lists.
- ☐ Evaluate whether templates actually reduce cognitive load.
- ☐ Move to higher-level orchestration where templates provide insufficient abstraction.

Template-based platforms can simplify provisioning, but excessive exposure to underlying cloud resource definitions can increase rather than reduce cognitive load.

6. Service Orchestration

Translate

- ☐ Translate higher-level platform specifications into lower-level resource definitions.
- ☐ Determine the appropriate level of abstraction.
- ☐ Support **composition** where one platform resource maps to multiple underlying resources.
- ☐ Support **lookup** where a logical request maps to a context-specific physical resource.
- ☐ Consider **model translation** when the platform exposes a different metamodel from the underlying cloud.

Deploy

- ☐ Use declarative provisioning where appropriate.
- ☐ Determine the difference between the desired and actual state.
- ☐ Apply the changes required to reconcile the two states.
- ☐ Handle resources that can be updated in place.
- ☐ Handle resources that must be replaced instead of updated.
- ☐ Ensure application artifacts are available when resource changes require code redeployment.

Service orchestration primarily consists of translating high-level requests and deploying the resulting target resource state.

7. Control Loop

- ☐ **Observe** changes in specifications and the environment.
- ☐ **Analyze** those changes against platform constraints.
- ☐ **Act** by applying the required changes.
- ☐ Repeat the control loop continuously.
- ☐ Detect configuration drift caused by manual changes.
- ☐ Define how detected drift will be resolved.
- ☐ Reconcile resources across cloud, SaaS, and installed application services.
- ☐ Manage identities and access relationships needed to interconnect services.

The document describes the platform control loop as **Observe** → **Analyze** → **Act**.

8. Traceability and Troubleshooting

- ☐ Maintain a link between generated resources and the higher-level platform construct that created them.
- ☐ Add tags or identifiers to generated resources when direct links are unavailable.
- ☐ Make errors traceable back to the relevant platform specification.
- ☐ Avoid abstractions that hide resources without providing a troubleshooting path.
- ☐ Ensure developers can identify the source of failures in generated infrastructure.

Traceability is essential because users need to connect runtime failures back to the higher-level abstraction that produced the affected resource.

9. Application Code and Architecture as Code

- ☐ Determine whether platform functionality requires custom application code.
- ☐ Provide build and deployment pipelines for platform-provided code.
- ☐ Integrate application delivery with infrastructure provisioning so the experience appears seamless.
- ☐ Consider modeling **application topology**, not only infrastructure hierarchy.
- ☐ Represent important data-flow and control-flow relationships.
- ☐ Evaluate whether **Architecture as Code (AaC)** is appropriate for the platform.
- ☐ Make platform-generated cost or latency implications visible to users.

Architecture as Code models application topology rather than merely representing a hierarchy of infrastructure resources.

10. Ownership and Responsibility

- ☐ Clearly define the responsibilities of the **platform team**.
- ☐ Clearly define the responsibilities of **application teams**.
- ☐ Give application teams enough configuration control to minimize unnecessary ticket-based friction.
- ☐ Retain platform-level restrictions required for security or regulatory compliance.
- ☐ Decide who owns provisioned resources.
- ☐ Ensure application teams receive the access rights required for their responsibilities.
- ☐ Preserve platform control over the resource lifecycle where necessary.
- ☐ Check whether platform-owned accounts could encounter resource or quota limits.

The source argues that platform speed comes partly from shifting more configuration responsibility to application teams while retaining necessary platform guardrails.

11. Tenancy Model

- ☐ Identify what constitutes a tenant:
 - ☐ Development team
 - ☐ Individual developer
 - ☐ Business unit
 - ☐ Other organizational unit
- ☐ Decide where tenancy should be implemented:
 - ☐ **Multi-Tenant Resource**
 - ☐ **Multi-Tenant**
 - ☐ **Independent Instances**
- ☐ Evaluate security isolation between tenants.
- ☐ Evaluate performance isolation between tenants.
- ☐ Protect the platform against **noisy-neighbor** problems.
- ☐ Consider operational overhead for onboarding each additional tenant.
- ☐ Determine whether underlying SaaS or cloud services can manage tenancy on the platform's behalf.
- ☐ Evaluate resource and quota limitations.
- ☐ Plan how updates will be propagated when many tenant-specific instances exist.

The document identifies three primary tenancy approaches: within the managed resource, within the developer platform, or as independent instances on the underlying cloud platform.

12. Data and Resource Isolation

Logical Separation

- ☐ Decide whether multiple tenants can safely share one logical resource.
- ☐ Ensure every access operation correctly applies the tenant identifier.
- ☐ Evaluate the risk of accidental cross-tenant data exposure.
- ☐ Check resource size and performance limits.

Namespace Separation

- ☐ Determine whether each tenant should receive a separate namespace, table, or similar subdivision.
- ☐ Configure tenant-specific access controls.
- ☐ Evaluate remaining noisy-neighbor risks.
- ☐ Check limits on the number of namespaces or tables.
- ☐ Automate schema or configuration changes across tenants.

Resource-Level Separation

- ☐ Determine whether each tenant requires a completely separate resource instance.
- ☐ Automate resource creation and deletion.
- ☐ Automate changes across all tenant-specific instances.
- ☐ Evaluate the additional operational overhead.
- ☐ Evaluate the cost implications of separate instances.

The database example progresses from logical separation to namespace separation and then resource-level separation, with increasing isolation but also greater management overhead.

13. Shared Resources and Fairness

- ☐ Identify resources vulnerable to noisy-neighbor behavior.
- ☐ Assess whether one tenant can monopolize shared capacity.
- ☐ Add quotas, limits, or isolation where needed.
- ☐ Prefer resource-level separation where fair sharing cannot be reliably enforced.
- ☐ Test behavior under uneven or bursty tenant workloads.

14. Hierarchy and Scale

- ☐ Determine whether the organization needs multiple levels of tenancy.
- ☐ Model business units, teams, and their resource hierarchies where required.
- ☐ Estimate the maximum number of resources the control plane must manage.
- ☐ Test control-plane behavior under large-scale reconciliation.
- ☐ Test bursts of resource callbacks after outages.
- ☐ Test large-scale version or configuration updates.
- ☐ Ensure the control plane will not become a bottleneck as platform adoption grows.

The source warns that uneven callbacks—for example, after an outage or during mass updates—can overload a control plane even when its normal operating scale is moderate.

Final Readiness Check

- ☐ Developers have an effective self-service interface.
- ☐ Platform abstractions genuinely reduce cognitive load.
- ☐ The desired and actual states are continuously reconciled.
- ☐ Services are cataloged, provisioned, and traceable.
- ☐ Generated resources can be traced back to their source abstraction.
- ☐ Application and platform-team responsibilities are explicit.
- ☐ The tenancy model has been consciously selected.
- ☐ Security and performance isolation have been validated.
- ☐ Cost and operational overhead are understood.
- ☐ Drift and failures can be diagnosed.
- ☐ The control plane has been tested for scale and failure scenarios.
- ☐ The platform can evolve without hiding critical implementation details from operators.